MINICURSO DE ROBÓTICA · PARTE 2

Robô Sentinela com Machine Learning

ML

Do sensor ao modelo embarcado: um robô que patrulha, percebe e reconhece gestos com uma árvore de decisão treinada por vocês.

Máquina de estados | Árvore de decisão | Python + Colab | TinyML

Prof. Paulo César da Silva Junior · Engenharia de Computação

ROTEIRO DO DIA

Parte 1 montou o corpo. Agora, o cérebro.

Horário	Bloco	O que fazemos	Código
0:00–1:40	Parte 1	Arduino passo a passo: corpo montado e testado	etapas 0–9
1:40–1:50	Intervalo	—	—
1:50–2:20	2 · Regras	Máquina de estados com millis()	sketch 02
2:20–3:15	3 · Machine Learning	Coleta, treino no Colab, avaliação	sketch 03 + Colab
3:15–3:50	4 · Integração	Modelo embarcado no robô	sketch 04
3:50–4:00	Encerramento	Teste cruzado entre grupos	—

O QUE VAMOS CONSTRUIR

Um sentinela que entende quatro gestos

0
PARADO

Mão parada ou ninguém. Continua vigiando.

1
APROXIMAR

Mão vindo em direção ao sensor. Dispara o alarme.

2
AFASTAR

Mão indo embora. Robô dá tchau e volta a patrulhar.

3
ACENAR

Vai-e-vem da mão. O robô cumprimenta de volta.

O robô **não** recebe essas regras prontas: ele aprende a diferenciar os gestos a partir de exemplos gravados por vocês.

BLOCO 2 · 1:50 – 2:20

Comportamento por regras

Programação sem travar o loop, máquinas de estados e o limite das regras escritas à mão.

BLOCO 2 · TEMPO REAL

O problema do delay(): o robô fica cego

Bloqueante

```
void loop() {
  cabeca.write(angulo);
  angulo += 4;
  delay(1000);  // 1 s parado
  float d = lerDistanciaCm();
}
```

Durante o delay o Arduino não lê sensor, não lê botão, não reage. Um intruso rápido passa sem ser visto.

Não bloqueante

```
void loop() {
  if (millis() - t < 60) return;
  t = millis();
  // só chega aqui a cada 60 ms
  float d = lerDistanciaCm();
}
```

O loop gira milhares de vezes por segundo e só "trabalha" quando chega a hora. O resto do tempo fica livre para outras tarefas.

BLOCO 2 · TEMPO REAL

millis(): o relógio do robô

```
const unsigned long PERIODO_MS = 50;
unsigned long tUltimo = 0;

void loop() {
  if (millis() - tUltimo < PERIODO_MS)
    return;
  tUltimo += PERIODO_MS; // ritmo fixo
  amostrar();
}
```

millis() devolve os milissegundos desde que a placa ligou, num **unsigned long** de 32 bits.

Ele zera após 2^{32} ms **≈ 49,7 dias** — e a subtração *agora − antes* continua certa, porque a aritmética sem sinal "dá a volta" junto.

+= PERIODO em vez de **= millis()**: o atraso de uma volta não se acumula. Amostras a cada 50 ms exatos.

BLOCO 2 · MÁQUINA DE ESTADOS

Um robô está sempre em algum estado

Estado

O que o robô está fazendo agora. Exatamente um por vez.

PATRULHA, ATENCAO, ALARME

Transição

A condição que faz trocar de estado.

"alvo a menos de 40 cm"

Ação

O que acontece ao entrar no estado, ou enquanto se está nele.

LED vermelho, bip, parar o servo

Por que usar? Troca um emaranhado de **if** aninhados por algo que cabe num desenho: dá para planejar no papel, testar estado por estado e explicar para outra pessoa.

BLOCO 2 · SKETCH 02

A sentinela por regras

PATRULHA
Servo varre 20°–160°
display P · LED verde

alvo < 40 cm (2 leituras) →

← sem alvo por 3 s

ATENÇÃO
Para e olha o alvo
display A · LED verm.

alvo < 12 cm →

← após 3 s

ALARME
Sirene + relé
LEDs vermelho e azul

No escuro (LDR abaixo do limiar), o relé — nosso "holofote" — acende já na ATENÇÃO. Ajuste os limiares no topo do sketch e observe a mudança.

enum + switch: cada estado, um case

```
enum Estado { PATRULHA, ATENCAO, ALARME };
Estado estado = PATRULHA;

switch (estado) {
  case PATRULHA:
    moverCabeca();
    if (contDeteccao ⩾ 2) mudarPara(ATENCAO);
    break;
  case ATENCAO:
    if (d < PERIGO_CM) mudarPara(ALARME);
    else if (sumiuHa3s) mudarPara(PATRULHA);
    break;
  case ALARME: /* sirene por 3 s */ break;
}
```

enum dá nomes legíveis aos estados (por baixo são 0, 1, 2).

Cada **if** dentro de um case é uma seta do diagrama.

mudarPara() concentra as ações de entrada: LEDs, display, bip, relé. Um só lugar para mudar.

BLOCO 2 · ROBUSTEZ

Não confie em uma leitura só

Confirmação

Só entra em ATENÇÃO após **2 leituras seguidas** abaixo de 40 cm. Um eco falso isolado é ignorado.

```
cont = (d < 40) ? cont + 1 : 0;
```

Histerese

A regra para **entrar** é diferente da regra para **sair**: entra rápido (2 leituras), sai devagar (3 s sem alvo).

Sem isso, um alvo na borda dos 40 cm faria o robô alternar de estado sem parar.

É o mesmo princípio do termostato do ar-condicionado: liga a 25 °C, só desliga a 23 °C.

DESAFIO · 10 MIN

Escreva as regras dos gestos

```
if ( ???  ) gesto = APROXIMAR;
else if ( ???  ) gesto = AFASTAR;
else if ( ???  ) gesto = ACENAR;
else gesto = PARADO;
```

A distância caiu quanto? Em quanto tempo?

E se a pessoa for mais lenta? Mais longe?

Quantas idas e voltas fazem um aceno?

Cada pergunta vira um limiar que alguém precisa adivinhar. E se o computador encontrasse os limiares sozinho, a partir de exemplos?

BLOCO 3 · 2:20 – 3:15

Machine Learning: o cérebro aprendido

Dados, features, árvores de decisão, treino e teste, overfitting — e como tudo isso cabe em 2 KB de RAM.

BLOCO 3 · A GRANDE IDEIA

Programar regras × aprender regras

PROGRAMAÇÃO TRADICIONAL (BLOCO 2)



MACHINE LEARNING (BLOCO 3)



Depois de aprendidas, as regras são usadas exatamente como no Bloco 2: entram dados novos, saem respostas. Essa fase se chama **inferência**.

BLOCO 3 · PANORAMA

Três formas de aprender

← O NOSSO

Supervisionado

Exemplos **com a resposta certa**. O modelo aprende a mapear entrada → rótulo.

Spam ou não; diagnóstico por imagem; nossos gestos.

SEM RÓTULOS

Não supervisionado

Só os dados, sem resposta. O modelo procura grupos e padrões sozinho.

Segmentar clientes; detectar anomalias.

TENTATIVA E ERRO

Por reforço

Um agente age, recebe recompensa ou punição e melhora com o tempo.

Robôs que aprendem a andar; jogos.

Dentro do supervisionado: **classificação** (a resposta é uma categoria — nossos 4 gestos) ou **regressão** (a resposta é um número — prever a temperatura de amanhã).

BLOCO 3 · VOCABULÁRIO

As palavras de quem trabalha com ML

Termo	Significado	No nosso robô
Amostra	Um exemplo observado	Um gesto gravado (1 s)
Feature	Característica numérica da amostra	Média, desvio, inclinação…
Rótulo	A resposta certa da amostra	0, 1, 2 ou 3
Dataset	Conjunto de amostras rotuladas	O CSV do grupo (~80 linhas)
Modelo	A função aprendida: features → classe	A árvore de decisão
Treino	Ajustar o modelo aos exemplos	No Colab, em segundos
Inferência	Usar o modelo em dados novos	No Arduino, a cada 1 s

BLOCO 3 · ANTES DE COLETAR DADOS

Sensores reais mentem às vezes

Superfícies difíceis

Tecido e espuma absorvem o som; superfícies inclinadas desviam o eco.

Eco que não volta

Sem timeout, pulseIn() espera até 1 s e o robô congela. Com 12 ms (~2 m), devolvemos 200 cm = "longe".

Leitura perdida no meio do gesto

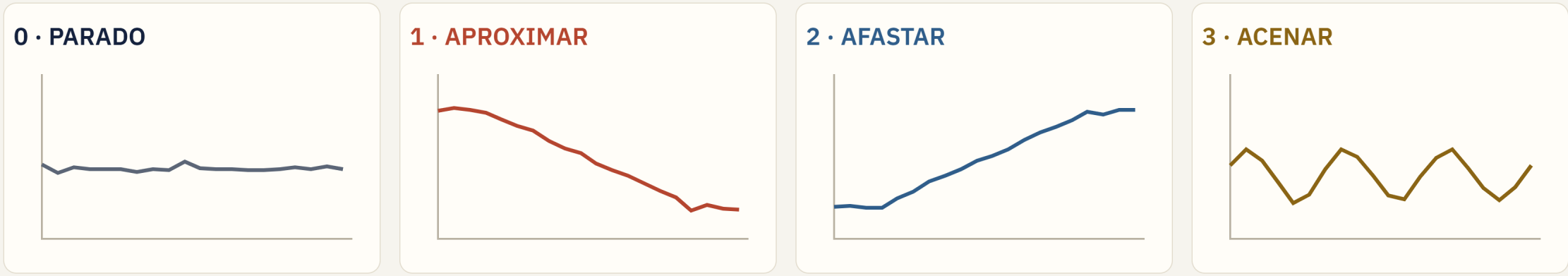
Um 200 isolado entre leituras de 20 cm é ruído: repetimos o valor anterior.

```
float lerDistanciaCm() {
  digitalWrite(TRIG, LOW);  delayMicroseconds(2);
  digitalWrite(TRIG, HIGH); delayMicroseconds(10);
  digitalWrite(TRIG, LOW);
  unsigned long us =
    pulseIn(ECHO, HIGH, 12000UL);
  if (us == 0) return 200.0;
  float d = us * 0.0343 / 2.0;
  return d > 200.0 ? 200.0 : d;
}
```

BLOCO 3 · DO SINAL À AMOSTRA

Um gesto = uma janela de 20 leituras

Lemos a distância a cada **50 ms** (20 Hz). Vinte leituras formam uma janela de **1 segundo**. Cada gesto deixa um desenho diferente:



Eixo vertical: distância (0 a 50 cm, mais longe em cima) · eixo horizontal: tempo (1 s) · curvas ilustrativas

BLOCO 3 · ENGENHARIA DE FEATURES

De 20 números para 6 que fazem sentido

Os 20 valores crus mudam se a mão estiver 5 cm mais perto ou começar um pouco depois. Resumimos a janela em grandezas que descrevem o **movimento**:

Feature	O que mede	Ajuda a separar
média	Distância média na janela (cm)	Tem alguém perto ou não
desvio	Quanto a distância variou	Parado × em movimento
mínimo, máximo	Menor e maior distância	Alcance do gesto
inclinação	Tendência da reta (cm/s)	APROXIMAR × AFASTAR
trocas	Vezes que a mão inverteu o sentido	ACENAR × o resto

BLOCO 3 · FEATURES ESTATÍSTICAS

Média e desvio padrão

$$\bar{x} = (1/n) \cdot \Sigma\, x_i$$

Onde a mão costuma estar.

$$\sigma = \sqrt{[\ (1/n) \cdot \Sigma\, (x_i - \bar{x})^2\ ]}$$

O quanto ela se espalha em torno da média. Raiz da média dos desvios ao quadrado.

Exemplo com 4 leituras

Janela (cm)	Média	Desvio
20, 22, 21, 19	20,5	1,1
10, 30, 10, 30	20,0	10,0

Médias quase iguais, desvios muito diferentes: a média sozinha não distingue mão parada de aceno.

BLOCO 3 · FEATURES DE MOVIMENTO

Inclinação e trocas de sentido

Inclinação (cm/s)

A reta que melhor passa pelos 20 pontos — **mínimos quadrados**:

$$b = \Sigma(t_i - \bar{t})(x_i - \bar{x}) \,/\, \Sigma(t_i - \bar{t})^2$$

Multiplicada por 20 amostras/s vira cm/s. De 40 para 10 cm em 1 s: **≈ −30 cm/s**. Negativa = aproximando.

Trocas de sentido

Para cada par de leituras, o sentido do movimento: **+1** se subiu mais de 2 cm, **−1** se desceu mais de 2 cm, **0** se ficou nessa faixa.

Conta quantas vezes o sentido inverte. Aceno: 2 a 6 trocas. Aproximar: 0.

A faixa de ±2 cm é uma "zona morta" contra o ruído do sensor.

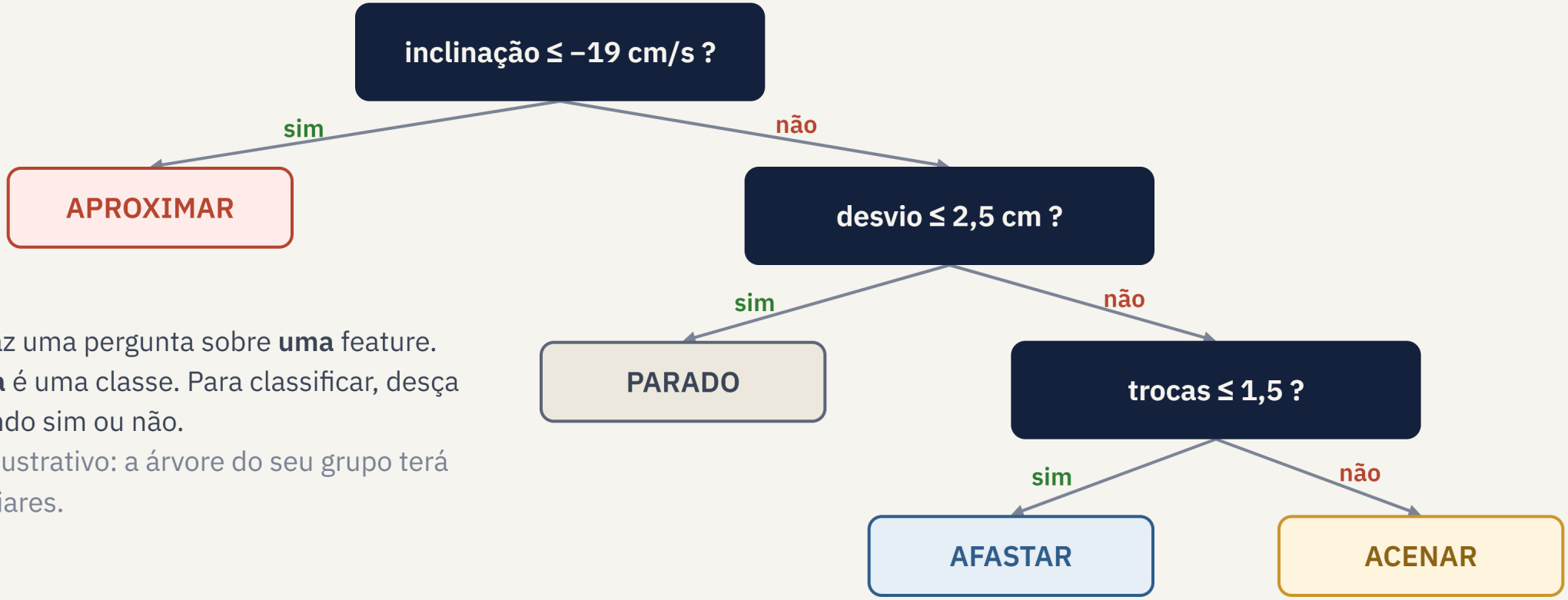
BLOCO 3 · INTUIÇÃO

Cada gesto tem uma ”assinatura”

Gesto	Inclinação	Desvio	Trocas
PARADO	≈ 0	baixo	0
APROXIMAR	muito negativa	alto	0 a 1
AFASTAR	muito positiva	alto	0 a 1
ACENAR	≈ 0	alto	2 ou mais

Mas "muito negativa" é quanto? –10? –25? E o desvio "alto" começa onde? **É exatamente isso que a árvore de decisão vai aprender com os dados.**

Árvore de decisão: perguntas em sequência



Cada **nó** faz uma pergunta sobre **uma** feature. Cada **folha** é uma classe. Para classificar, desça respondendo sim ou não.

Exemplo ilustrativo: a árvore do seu grupo terá outros limiares.

BLOCO 3 · COMO A ÁRVORE APRENDE

Escolher a pergunta que mais ”purifica”

$$Gini = 1 - \Sigma\ p_k^2$$

p_k = fração de exemplos da classe k no grupo. Gini 0 = grupo puro; quanto maior, mais misturado.

1. Testa toda feature e todo limiar possível
2. Escolhe o que mais reduz a impureza
3. Repete em cada lado até ficar puro ou atingir max_depth

Exemplo: 10 APROXIMAR + 10 AFASTAR

Antes: $1 - (0{,}5^2 + 0{,}5^2) =$ **0,50**

Pergunta "inclinação ≤ 0?":

sim → 10 APROX + 1 AFAST → Gini 0,17
não → 9 AFAST → Gini 0,00

Média ponderada: $11/20 \times 0{,}17 + 9/20 \times 0 \approx$ **0,09**

Queda de 0,50 para 0,09: ótima pergunta.

BLOCO 3 · AVALIAÇÃO HONESTA

Nunca avalie com a lista de exercícios



Divisão estratificada: cada parte mantém a mesma proporção das 4 classes.

O modelo **aprende** só com o treino. O teste fica guardado, como uma prova com questões que o aluno nunca viu.

Acertar no treino é fácil — basta decorar. A pergunta que importa: **acerta em gestos novos?**

```
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.25, stratify=y)
```

BLOCO 3 · MÉTRICAS

Matriz de confusão: onde o modelo erra

Real ↓ / Previsto →	PARADO	APROX.	AFAST.	ACENAR
PARADO	12	0	1	0
APROXIMAR	0	11	0	1
AFASTAR	1	0	12	0
ACENAR	0	0	1	11

Números ilustrativos, 50 gestos de teste. Diagonal = acertos.

92%

Acurácia = acertos / total = 46 / 50

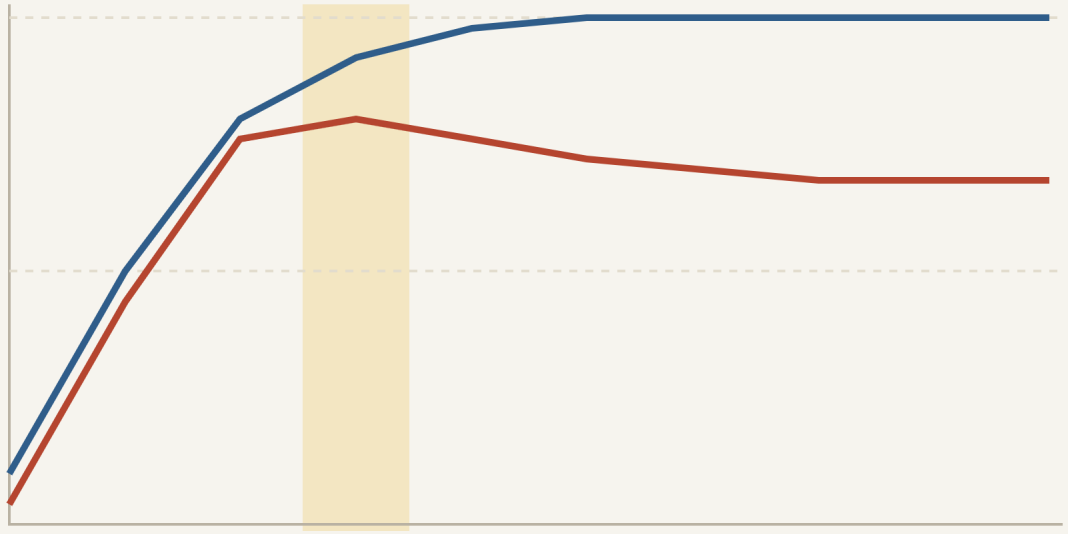
Precisão de AFASTAR: dos 14 que o modelo chamou de AFASTAR, 12 eram — 86%.

Revocação de AFASTAR: dos 13 AFASTAR reais, achou 12 — 92%.

BLOCO 3 · O ERRO MAIS IMPORTANTE

Overfitting: decorar não é aprender

— acurácia no treino — acurácia em dados novos



profundidade da árvore (max_depth) de 1 a 10 → · ilustrativo

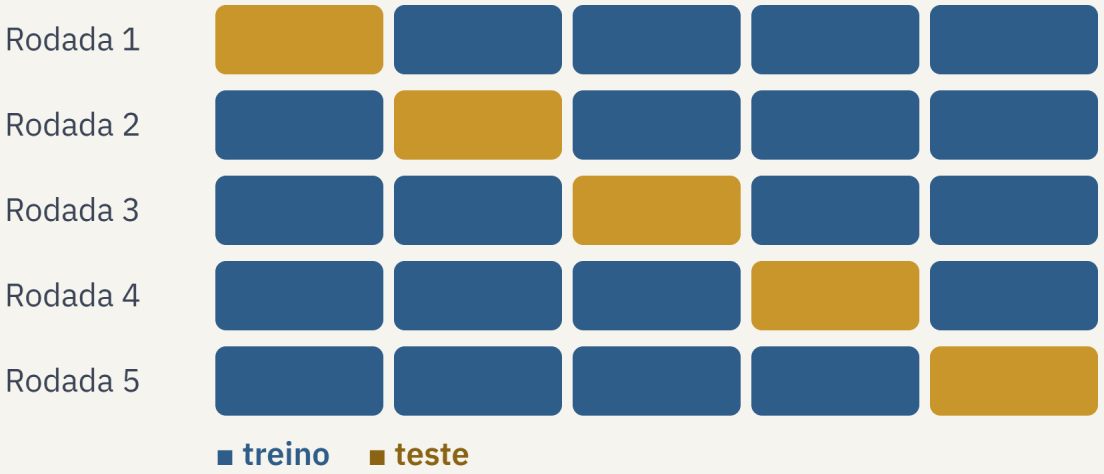
Rasa demais (underfitting): poucas perguntas, não captura a diferença entre gestos.

Ponto ideal: a acurácia em dados novos é máxima.

Funda demais (overfitting): cria regras para cada exemplo, inclusive para o ruído. 100% no treino, pior no mundo real.

BLOCO 3 · VALIDAÇÃO CRUZADA

Com poucos dados, teste cinco vezes



Os dados viram **5 partes**. Cada rodada testa numa parte diferente e treina nas outras 4.

A nota final é a **média das 5**: todo gesto foi usado como teste uma vez.

É assim que o notebook escolhe a melhor profundidade.

BLOCO 3 · GENERALIZAÇÃO

O modelo só sabe o que os dados mostraram

Dataset viciado

Todos os gestos feitos pela mesma pessoa, na mesma velocidade, a 20 cm. O modelo vai muito bem com ela... e falha com o colega.

Dataset diverso

Todos do grupo gravam. Gestos rápidos e lentos, perto e longe, mão aberta e fechada, nada na frente. Mais difícil de decorar, melhor no mundo real.

No notebook, o teste **"deixa um grupo de fora"** treina com os dados de 3 grupos e testa no 4º: a forma mais honesta de saber se o robô funciona com gente nova.

Por que uma árvore cabe em 2 KB

Modelo	O que precisa guardar e calcular	No Arduino Uno
Árvore (prof. 4)	Só os limiares, dentro do código; no máximo 4 comparações	~0 B de RAM extra · cabe com folga
k-NN	Todos os exemplos: 80 × 6 × 4 B = 1.920 B	94% da RAM só para os dados
Rede neural 6→16→4	180 pesos (720 B) e ~160 multiplicações	Possível, mas apertado

TinyML é rodar modelos em microcontroladores: sem internet, em microssegundos e com miliwatts. Placas maiores (ESP32, Nano 33 BLE) já rodam redes neurais com TensorFlow Lite Micro.

MÃO NA MASSA · 15 MIN

Coleta de dados: sketch 03

1. Upload do 03_coleta_dados; Serial Monitor a 9600
2. Digite a classe (0 a 3) + Enter: o display mostra
3. Aperte GRAVAR: bip grave = prepare-se; bip agudo + LED azul = gravando 1 s
4. ~20 gravações por classe, todos do grupo
5. Copie todo o Serial Monitor e salve como dados_grupoX.csv

Classe	Como fazer
0 PARADO	Metade mão parada a 10–30 cm; metade sem nada
1 APROXIMAR	De ~40 cm até ~10 cm durante o LED azul
2 AFASTAR	De ~10 cm até ~40 cm
3 ACENAR	Vai-e-vem rápido entre ~10 e ~30 cm

Cada linha do CSV: **media, desvio, minimo, maximo, inclinacao, trocas, classe** — as features já são calculadas no Arduino.

MÃO NA MASSA · 20 MIN

Treino no Google Colab

1. colab.research.google.com → Arquivo → Fazer upload de notebook → treino_modelo_colab.ipynb
2. Executar as células em ordem (Shift+Enter)
3. Na célula 1, enviar o dados_grupoX.csv
4. Treinar com PROFUNDIDADE = 4
5. Rodar o experimento de profundidade 1 a 10
6. Última célula: baixa o **modelo.h**

O que anotar na ficha

- Qual feature mais separa as classes nos gráficos?
- Acurácia de treino e de teste
- Qual par de gestos mais se confunde na matriz?
- Melhor profundidade pela validação cruzada

BLOCO 3 · DO PYTHON PARA O C

A árvore vira um arquivo modelo.h

```
int classificar(const float *f) {
  if (f[4] ⩽ -19.0f) {      // inclinacao
    return 1;              // APROXIMAR
  } else {
    if (f[1] ⩽ 2.5f) {     // desvio
      return 0;           // PARADO
    } else {
      if (f[5] ⩽ 1.5f) return 2; // AFASTAR
      else             return 3; // ACENAR
    }
  }
}
```

O notebook percorre a árvore: cada **nó** vira um **if/else**, cada **folha** vira um **return**.

f[0]...f[5] são as 6 features, na mesma ordem do CSV.

Gerado automaticamente — nenhum aluno escreve esses limiares à mão.

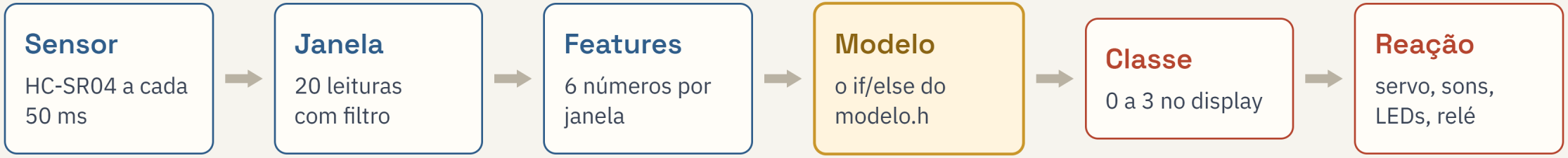
BLOCO 4 · 3:15 – 3:50

Integração: o robô ganha cérebro

O modelo treinado entra no firmware. Testar, entender os erros, coletar mais, retreinar.

BLOCO 4 · VISÃO COMPLETA

O caminho de um gesto até a reação

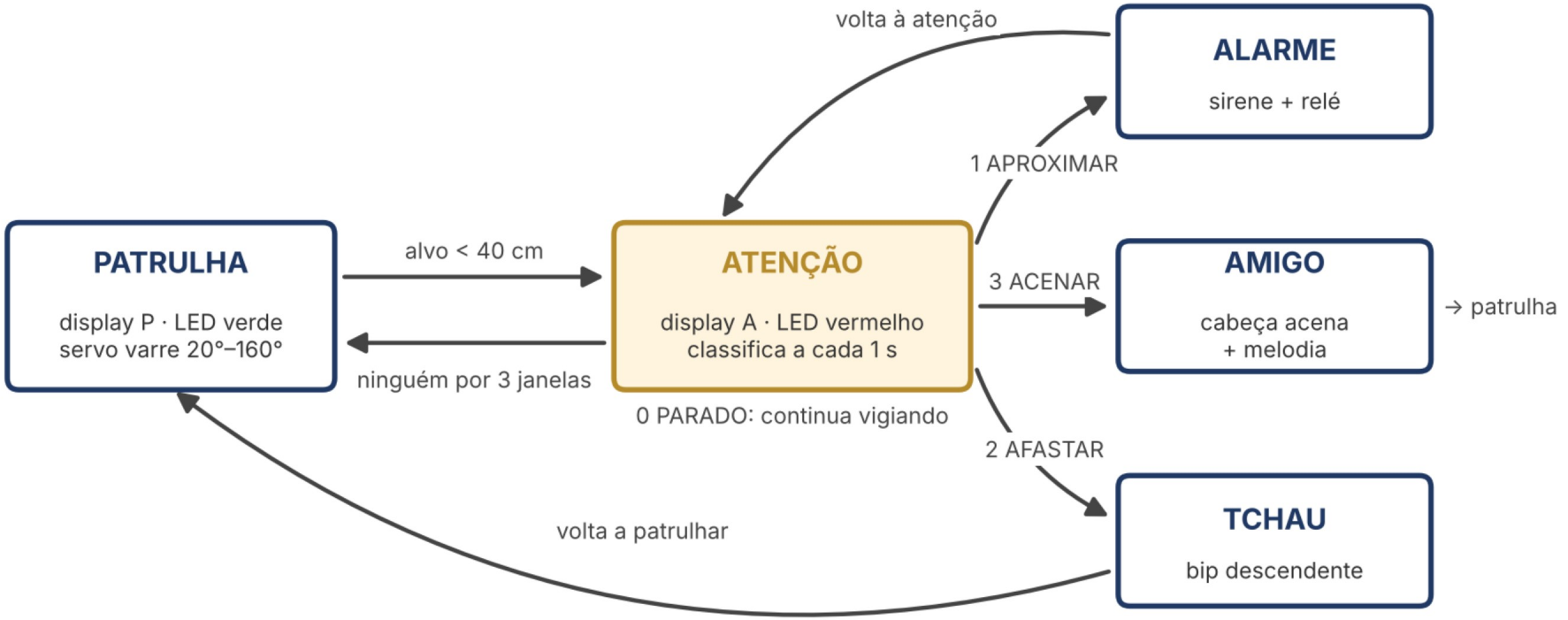


Tudo isso roda no Arduino, uma vez por segundo, enquanto o robô está em ATENÇÃO. Nenhuma conexão com o computador é necessária.

Só o **treino** aconteceu fora, no Colab. O que veio de lá foi apenas o arquivo modelo.h, compilado junto com o sketch 04.

BLOCO 4 · SKETCH 04

A máquina de estados, agora com o modelo



BLOCO 4 · A REGRA DE OURO

Treino e robô precisam medir igual

Item	Sketch 03 (coleta)	Sketch 04 (robô)
N_AMOSTRAS	20	20
PERIODO_MS	50	50
Filtro de leitura perdida	repete a anterior	repete a anterior
calcularFeatures()	original	cópia exata

⚠ Mude o período para 100 ms só no robô e a janela vira 2 s: o desvio e as trocas mudam, e o modelo passa a receber números que nunca viu. Ele não avisa — só erra.

MÃO NA MASSA · 30 MIN

Embarcar, testar, melhorar



1. Copiar o modelo.h para a pasta 04_sentinela_ml
2. Conferir os ajustes do topo (display, buzzer, relé, LDR)
3. Upload e Serial Monitor aberto
4. Esperar o bip da ATENÇÃO e fazer o gesto

Errou muito um gesto?

Volte ao sketch 03, grave mais exemplos dessa classe, retreine e embarque de novo. É assim que se trabalha com ML de verdade.

BLOCO 4 · DEPURAÇÃO

Quando algo dá errado

Sintoma	Causa provável	O que fazer
Distância sempre 200	TRIG/ECHO trocados	Conferir D7 = TRIG, D8 = ECHO
Arduino reinicia ao mexer o servo	Queda de tensão	Capacitor 100 µF; USB direta
Display "em negativo"	Tipo de display	Inverter DISPLAY_ANODO_COMUM
'modelo.h: No such file'	Arquivo fora da pasta	modelo.h ao lado do .ino
Colab: 0 amostras carregadas	CSV sem linhas de dados	Conferir linhas com 7 números
Todo gesto vira APROXIMAR	Gesto antes do bip	Esperar o bip da ATENÇÃO
Confunde dois gestos	Poucos exemplos variados	Gravar mais dessa classe

ENCERRAMENTO · 10 MIN

Teste cruzado: o robô entende um estranho?

1

Cada grupo apresenta em 2 minutos: acurácia no Colab e demonstração.

2

Um aluno de **outro grupo** faz 8 gestos no robô. Contem os acertos.

3

Compare com o acerto do próprio grupo. Caiu? Por quê?

Os dados de treino definem o que o modelo sabe. Para funcionar com qualquer pessoa, precisamos de dados de muitas pessoas.

REVISÃO

Em 4 horas, vocês passaram por

Eletrônica embarcada

E/S digital e analógica, Lei de Ohm, ultrassom, PWM do servo, timers, alimentação.

Software de tempo real

Loop sem delay com millis(), máquina de estados, confirmação e histerese.

Ciclo de Machine Learning

Dataset rotulado, features, árvore de decisão, treino × teste, overfitting, validação cruzada.

TinyML

Modelo treinado em Python, convertido em C e executado num chip de 2 KB de RAM.

PARA CONTINUAR

Desafios para levar o robô adiante

No próprio robô

- Nova classe: "dois acenos" ou "mão bem perto"
- Modelo da turma: treinar com os 4 CSVs
- Comparar a árvore com k-NN no notebook
- Radar: plotar ângulo × distância da patrulha
- Usar o NTC (A1) para corrigir a velocidade do som

Onde isso volta no curso

- Circuitos Digitais: máquinas de estados
- Microcontroladores e Sistemas Embarcados
- Probabilidade e Estatística: média, desvio, regressão
- Processamento de Sinais: amostragem e Nyquist
- Inteligência Artificial e Aprendizado de Máquina

ROBÔ SENTINELA · MINICURSO DE ROBÓTICA

Obrigado!

Perguntas? O robô, os códigos e o notebook agora são de vocês.

ML

Prof. Paulo César da Silva Junior · Engenharia de Computação